Task-9:- Requirement and Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to outline the functional and non-functional requirements for deploying the PowerOfMath application on AWS using two distinct methodologies: Serverless and VM-based. The intended audience includes DevOps engineers, cloud architects, and stakeholders interested in architectural cost-performance trade-offs.

1.2 Scope

- **In-Scope:** Deployment of the web application, configuration of AWS Lambda, API Gateway, S3, EC2, Nginx, and SSL/HTTPS integration.
- **Out-of-Scope:** Development of new application features, database migration, or multi-region failover configuration.

1.3 Definitions, Acronyms, and Abbreviations

- **VM:** Virtual Machine (AWS EC2)
- **Serverless:** Execution model where the provider manages server allocation (AWS Lambda)
- **ACM:** AWS Certificate Manager
- **S3:** Simple Storage Service

1.4 References

- GitHub Repository: powerofmath
- AWS Well-Architected Framework Documentation

1.5 Overview

This document covers the project objectives, specific use cases for math calculations, non-functional constraints like security and reliability, and a risk analysis for both deployment strategies.

---

2. Project Overview

2.1 Project Objectives

- Deploy a functional math application on AWS.
- Demonstrate proficiency in both modern (Serverless) and traditional (IaaS) cloud patterns.
- Implement end-to-end security via SSL.

2.2 Business Goals

- **Cost Optimization:** Compare the "pay-as-you-go" serverless model vs. the "always-on" VM model.
- **Operational Excellence:** Minimize manual intervention through automation and managed services.

2.3 Background

The PowerOfMath application requires a simple frontend to take user input (base and exponent) and a backend to return the result. As the user base scales, the infrastructure must handle varying loads.

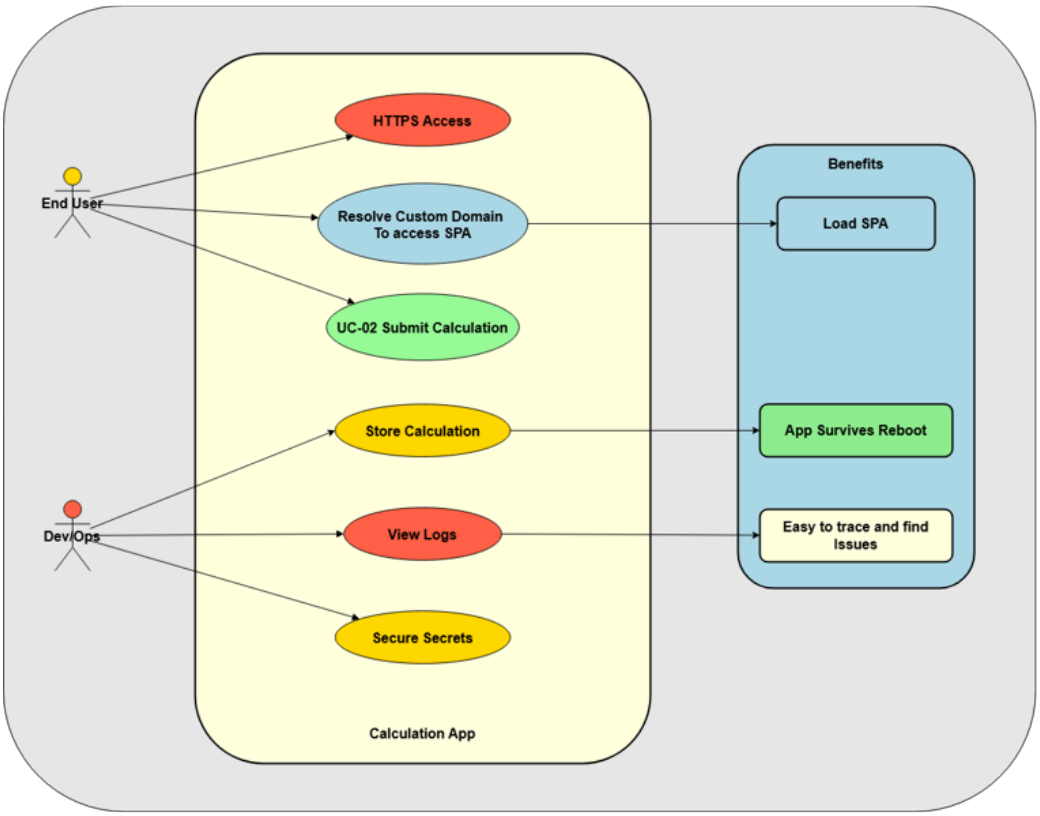
2.4 Constraints

- **Budget:** Must stay within the AWS Free Tier where possible.
- **Time:** Deployment must be completed within the specified task window.
- **Technology:** Limited to AWS Cloud services.

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Calculate Power of a Number

- **Description:** User inputs two numbers to calculate the result of $Base^{Exponent}$.
- **Actors:** End User, Web Browser.
- **Preconditions:** Application URL is accessible; AWS services are "Up."
- **Postconditions:** User receives the correct mathematical result.
- **Flow of Events:**
  a. User enters values into the web form.
  b. User clicks "Calculate."
  c. Frontend sends a POST request to the API (Gateway or Nginx).
  d. Backend processes logic and returns a JSON response.
  e. Result is displayed on the UI.
- **Alternative Flows:**
  a. Invalid input (strings instead of numbers): System returns a 400 Error.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- **Response Time:** Math calculations should return results in < 2 seconds.
- **Concurrency:** Serverless must handle up to 1000 concurrent executions (default limit).

4.2 Security Requirements

- **Encryption:** All traffic must be encrypted via TLS 1.2+ (HTTPS).
- **IAM:** Principle of Least Privilege for Lambda and EC2 roles.

4.3 Usability Requirements

- **Responsive UI:** The web interface should be usable on both desktop and mobile browsers.

4.4 Reliability Requirements

- **Uptime:** Target 99.9% availability for the API Gateway and S3.
- **Recovery:** EC2 must have a systemd service to auto-restart the app on crash.

4.5 Compliance Requirements

- **Data Privacy:** No PII (Personally Identifiable Information) is collected or stored.

---

5. Assumptions and Dependencies

5.1 Assumptions

- The GitHub repository code is stable and production-ready.
- The user has an active AWS account with administrative permissions.

5.2 Dependencies

- **Domain Name:** Required for SSL/ACM certificate validation.
- **Internet Connectivity:** Required for both the deployment process and end-user access.

---

6. Risk Analysis

6.1 Risk Identification

1. **Cost Spikes:** High traffic on Lambda could lead to unexpected costs.
2. **Security Breach:** Open ports on EC2 (SSH/HTTP) are targets for brute force.

6.2 Risk Mitigation

1. Set AWS Cost Budgets and Alerts.
2. Use Security Groups to restrict SSH access to specific IP addresses only.

---

7. Glossary

- **Reverse Proxy:** A server (Nginx) that sits in front of backend applications and forwards client requests.
- **Cold Start:** The delay experienced when a serverless function is triggered after being idle.

---

8. Appendices

- Architecture Diagram (To be attached in Technical Documentation).
- Code snippets for `nginx.conf` and `lambda_handler.py`.

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____